SAVEETHA SCHOOL OF ENGINEERING
C PROGRAMMING ASSIGNMENT – Advanced Problem Solving and Algorithm Design

Field	Entry
Name	BAVADARANI S
Reg. Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Advanced Problem Solving and Algorithm Design (Q1–Q7, 50 marks)
GitHub Repository URL	https://github.com/bavadarini28 / CSA0209-1912260562-BAVADARANI.S.git
Submission Date	12 NOVEMBER 2026

Q5 – Processing Text Without the Library

–Complete the Code (TODO 1 and TODO 2)

```
int word_count(const char s[])
{
   int count = 0, inside = 0;
   for (int i = 0; s[i] != '\0'; i++) {
      if (s[i] != ' ') { if (!inside) { count++; inside = 1; } }   /* word starts */
      else inside = 0;
   }
   return count;
}

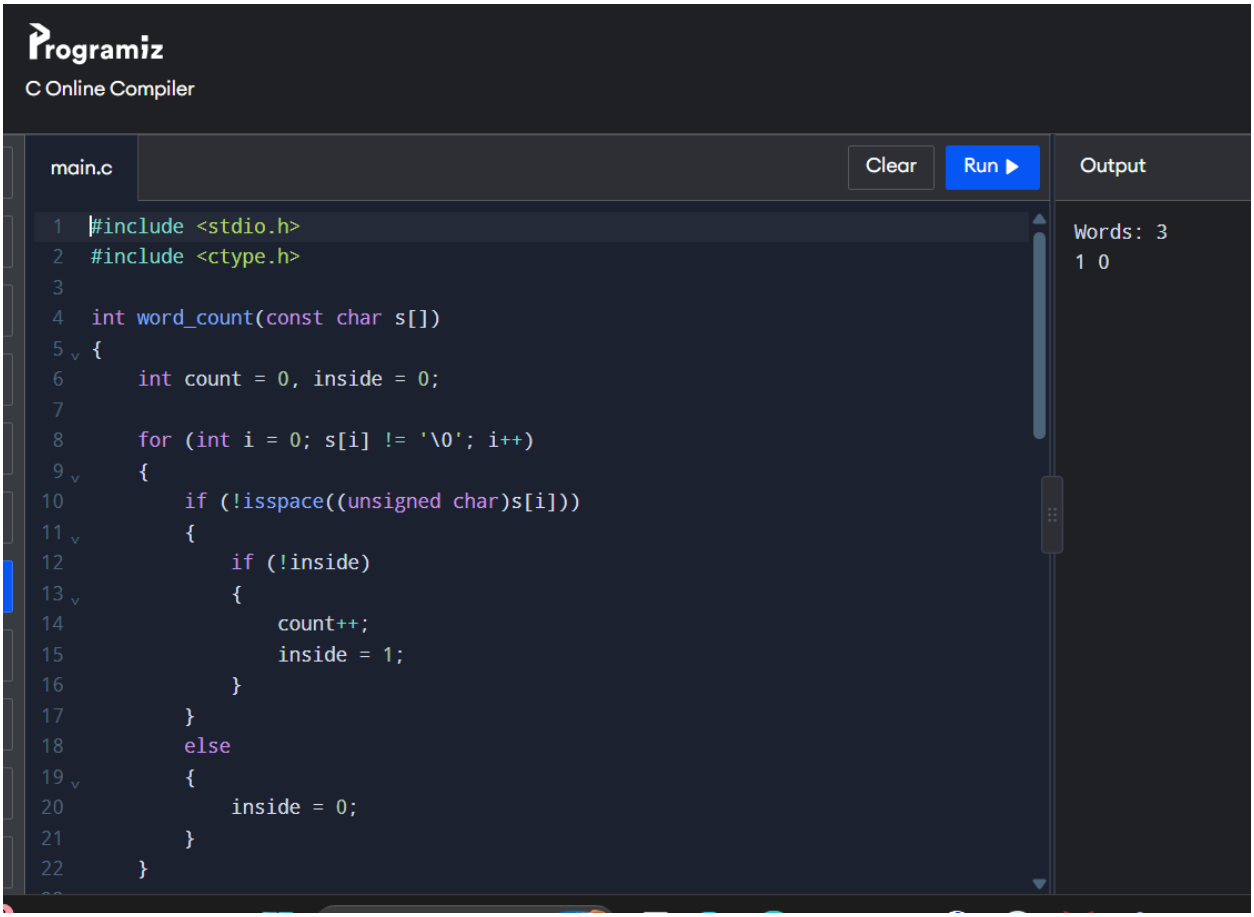
int is_palindrome(const char s[])
{
   int left = 0, right = 0;
   while (s[right] != '\0') right++;
   right--;
   while (left < right) {
      while (left < right && !isalpha((unsigned char)s[left]))  left++;
      while (left < right && !isalpha((unsigned char)s[right])) right--;
      if (tolower((unsigned char)s[left]) != tolower((unsigned char)s[right])) return 0;
      left++; right--;
   }
   return 1;
}
```

Output (matches expected):

```
Words: 7
1 0
```

Other tests: empty message → 0 words and palindrome result 1 (an empty string reads the same both ways, a convention that should be documented); "hello world" → upper-case HELLO WORLD → reversed DLROW OLLEH. The same functions written with strlen, strcpy, strcmp and toupper gave identical results (e.g. toupper version of "Level" → LEVEL, strlen 5, strcmp of two equal strings 0).

Tab test: word_count("go\tnow") returned 1 with the space-only test, because a tab is a non-space character, so "go\tnow" looks like one long word. Replacing s[i] != ' ' with !isspace((unsigned char)s[i]) (from ctype.h) treats tabs and newlines as separators and returns 2.



Student – Character-Level Design

- **Role of '\0':** a C string is a char array that ends with the null character (value 0). It is how every string function knows where the text stops, since the array has no stored length. The array must have room for it: "Meet" needs 5 bytes (measured sizeof = 5).

```
COUNT_WORDS(s):  count = 0 ; inside = 0
  for each character ch of s until '\0':
    if ch is not a space:
      if inside == 0: count = count + 1 ; inside = 1
    else: inside = 0
  return count          /* several spaces simply keep inside = 0 */
```

- **Why not recompute the length in every iteration:** strlen scans the whole string every time, O(n) per call, so a loop that calls it in its condition costs $O(n^2)$. Measured for n = 200,000 characters at -O0: 379.5 ms with strlen in the condition against 0.63 ms with the length stored once (about 600× faster).
- **char s[] = "OK" versus char *s = "OK":** the first is a 3-byte array in writable memory, initialised from the literal (sizeof 3), so s[0] = 'o' is legal. The second is a pointer (sizeof 8 on this system) to a string literal that may sit in read-only memory, so modifying it is undefined behaviour and usually crashes.

Student – Library Functions and Safety

strcpy overflow: copying a 44-character message into char small[8] gives, at compile time, "warning: writing 45 bytes into a region of size 8 overflows the destination" and, at run time:

```
*** stack smashing detected ***: terminated
Aborted    (exit status 134)
```

Without the compiler's protection the extra bytes would overwrite neighbouring variables and the return address, a classic security hole. Safe replacements, tested with the same message (output: "This me", 7 characters plus the terminator):

```
strncpy(small, msg, sizeof small - 1); small[sizeof small - 1] = '\0';   /* must terminate manually */
snprintf(small, sizeof small, "%s", msg);                          /* always terminates */
fgets(buf, sizeof buf, stdin); buf[strcspn(buf, "\n")] = 0;          /* bounded input, strip newline */
```

Why gets() was removed: gets() cannot be told the size of the buffer, so any input longer than the array overflows it and the program cannot defend itself. It was deprecated in C99 and removed in C11; fgets takes a size argument and replaces it.

Joint conclusion: write your own function when learning, when the task needs a rule the library lacks (words separated by several spaces, ignoring non-letters in a palindrome) or when no library exists (some embedded systems). Rely on string.h for standard operations (copy, compare, length): it is tested, optimised and consistent – but only with bounded versions and buffers of known size.

Alternative Concept and Complexity Analysis

for (i = 0; i < strlen(s); i++) calls strlen n + 1 times, each O(n): $O(n^2)$ in total (379.5 ms against 0.63 ms for n = 200,000). Storing the length first, or testing s[i] != '\0', is O(n). (An optimising compiler may hoist strlen out of the loop when it can prove the string does not change, but the code should not rely on that.) A recursive reversal – swap s[i] and s[j], then call itself for i + 1, j − 1 – is also O(n) time but uses n/2 + 1 stack frames (measured depth 6 for 10 characters), i.e. O(n) stack space against O(1) for the loop; at a few dozen bytes per frame a string of a million characters could exhaust the default 8 MB stack. For SMS-length messages (160 characters) both are fine; the loop is safer.